

Fluxo de Autenticacao - JP Cashflow

Fluxo de Autenticacao

(WEB) Fluxo de Navegação - JP Cashflow (Middleware -> Página Inicial)

1. Middleware (`middleware.ts`)

- Objetivo: Identificar o subdomínio e redirecionar corretamente.
- Execução: Antes de qualquer página.

Etapas:

1. Captura o host via `request.headers.get("host")`.
2. Extrai o subdomínio (jp de `jp.localhost:3000`).
3. Se o path for "/", redireciona para `"/inicio"`.
4. Se for `"/login"` e tiver token, redireciona para `"/dashboard"`.
5. Para outras rotas com subdomínio, faz `rewrite` da URL (mantém o caminho).
6. Se não houver subdomínio, continua normalmente (`NextResponse.next`).

2. Página Inicial (`src/app/(app)/inicio/page.tsx`)

- Objetivo: Verificar subdomínio, validar o login, e redirecionar corretamente.
- Execução: Após redirecionamento de "/" para `"/inicio"`.

Etapas:

1. Extrai o subdomínio do host.
2. Chama a API `/validaSubdominio` com POST para verificar se o login existe.
3. Se o login não for válido, redireciona para o domínio público `/`.
4. Se houver token nos cookies:
 - Decodifica o token (`decodeToken`).
 - Verifica se o login do token corresponde ao subdomínio:
 - Se corresponder, redireciona para `/dashboard`
 - Se for diferente, redireciona para `/login?user=subdomínio` (força logout).
5. Se não houver token, redireciona para `/login?user=subdomínio`.

Fluxo de Autenticacao - JP Cashflow

3. Página de Login (`src/app/(app)/login/page.tsx`)

- Objetivo: Verificar se o usuário já está autenticado e garantir que ele só acesse seu próprio ambiente.

Etapas:

1. Lê o cookie token e decodifica com `decodeToken`.
2. Lê o parâmetro user da URL (`?user=JP`) e converte para caixa alta.
3. Se já estiver logado e `?user` não informado:
 - Evita loop -> redireciona para `/dashboard`.
4. Se estiver logado e o login do token for igual ao user da URL:
 - Redireciona para `/dashboard`.
5. Se estiver logado com outro usuário:
 - Retorna o componente `<ForceLogout user={urlUser} />` para forçar logout.
6. Se não estiver logado:
 - Exibe o formulário de login (`<LoginForm defaultLogin={urlUser} />`).

4. Forçar Logout (`ForceLogout.tsx`)

- Objetivo: Forçar o logout quando o token não corresponde ao subdomínio acessado.

Etapas:

1. Remove o cookie token com `document.cookie = "token=; Max-Age=0;"`.
2. Limpa o contexto global: `usuariold`, `usuarioLogin`, `usuarioNome`, `usuarioPerfil`, `emailVerificacao`, `codigoVerificacao`.
3. Redireciona para `/login?user=subdomínio` usando `router.replace`.

5. Página do Dashboard (`/dashboard/page.tsx` e `DashboardClient.tsx`)

- Objetivo: Validar o token e exibir os gráficos e cards estatísticos.

Etapas:

1. Server Component:
 - Lê o cookie token e valida via `decodeToken`.
 - Se inválido ou ausente -> redireciona para `/login`.
 - Se válido -> renderiza o componente `<DashboardClient />`.

Fluxo de Autenticacao - JP Cashflow

2. Client Component (DashboardClient):

- Envolva o conteúdo com DashboardProvider.
- Renderiza os componentes visuais:
 - Gráficos: GraficoBarDespesas, GraficoPizzaEntradas, GraficoBarSubContas
 - Controles: SelectContas
 - Cards: CardConta (Despesas, Receitas, Cartões, Saldo, Investimentos)

6. Gráficos do Dashboard (graficoBarDespesas, etc.)

- Objetivo: Exibir dados de forma visual com base nos endpoints protegidos.

Etapas gerais por componente:

1. Ao montar, o gráfico chama uma função como RetEstatisticaDespesas(periodoId).
2. Essa função busca o token do cookie (client-side) com getTokenFromCookie.
3. A requisição fetch para API privada é feita com Authorization: Bearer <token>.
4. Os dados recebidos são colocados no useDashboardContext, disparando re-render.